ANCHOR UNIVERSITY
LAGOS

Name: Ajayi Oluwapelumi Joseph

Matriculation Number: AUL/CMP/22/013

Course Code: CMP 414

Course Title: Operating Systems II

Question 1: Using FCFS, given the following process table:

Process	Arrival Time	Burst Time	Turnaround Time
P1	0	5	5
P2	5	7	7
P3	10	6	8

Find the following:

- Completion time
- Average Completion time
- Inter-arrival time
- Average Inter-Arrival Time
- Waiting time
- Average waiting time

Question 2: Write a code for synchronization.

Question 3: Write a code for one single producer and consumer.

Question 1

```
#include <stdio.h>


int main() {
    int n, i;
    int bt[10], at[10], ct[10], wt[10], tat[10], iat[10];
    float avg_ct = 0, avg_iat = 0, avg_wt = 0, avg_tat = 0;

    printf("CMP 414 Assignment Solver\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    // Input Phase
    for (i = 0; i < n; i++) {
        printf("\nProcess P[%d]\n", i + 1);
        printf("Enter Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Enter Burst Time: ");
        scanf("%d", &bt[i]);
    }

    // Calculation Phase
    int current_time = 0;

    for (i = 0; i < n; i++) {
        // Calculate Inter-arrival time (Gap between this arrival and previous)
        if (i == 0) {
            iat[i] = 0; // First process has no previous arrival
        } else {
```

```
        iat[i] = at[i] - at[i-1];
    }

    // Logic: CPU waits if process hasn't arrived yet
    if (current_time < at[i]) {
        current_time = at[i];
    }

    // Completion Time = Start Time + Burst Time
    ct[i] = current_time + bt[i];

    // Update current time for next process
    current_time = ct[i];

    // Turn Around Time = Completion Time - Arrival Time
    tat[i] = ct[i] - at[i];

    // Waiting Time = Turn Around Time - Burst Time
    wt[i] = tat[i] - bt[i];

    // Accumulate for averages
    avg_ct += ct[i];
    avg_iat += iat[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
}

// Averages
avg_ct /= n;
```

```
    avg_iat /= (n - 1); // Usually averaged over n-1 gaps, but can be n depending on professor
    avg_wt /= n;
    avg_tat /= n;

    // Output Result
    printf("\n\n--- Results ---\n");
    printf("P\tArr\tBurst\tInterArr\tComp\tWait\tTurnAr\n");
    for (i = 0; i < n; i++) {
        printf("P[%d]\t%d\t%d\t%d\t\t%d\t%d\t%d\n",
               i+1, at[i], bt[i], iat[i], ct[i], wt[i], tat[i]);
    }

    printf("\nAverage Completion Time: %.2f", avg_ct);
    printf("\nAverage Inter-Arrival Time: %.2f", avg_iat); // (Note: First process gap is usually ignored)
    printf("\nAverage Waiting Time: %.2f\n", avg_wt);

    return 0;
}
```

Output

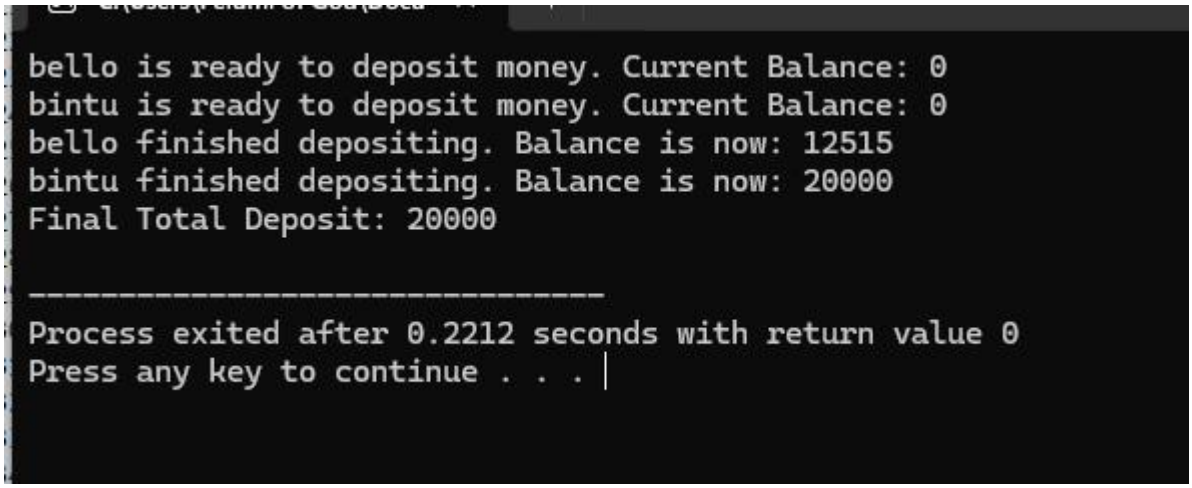
```
CMP 414 Assignment Solver
Enter number of processes: 5

Process P[1]
Enter Arrival Time: 1
Enter Burst Time: 2

Process P[2]
Enter Arrival Time: 3
Enter Burst Time: 4

Process P[3]
Enter Arrival Time: 5
Enter Burst Time: 6

Process P[4]
Enter Arrival Time: 7
Enter Burst Time: 8

Process P[5]
Enter Arrival Time: 9
Enter Burst Time: 0


--- Results ---
P       Arr     Burst   InterArr        Comp    Wait    TurnAr
P[1]    1       2       0               3       0       2
P[2]    3       4       2               7       0       4
P[3]    5       6       2               13      2       8
P[4]    7       8       2               21      6       14
P[5]    9       0       2               21      12      12

Average Completion Time: 13.00
Average Inter-Arrival Time: 2.00
Average Waiting Time: 4.00
```

Question 2

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>

pthread_mutex_t myMutex;
static volatile int balance = 0;

void *deposit(void *parameter) {
    char *who = (char *)parameter;
    int i;

    printf("%s is ready to deposit money. Current Balance: %d\n", who, balance);

    for (i = 0; i < 10000; i++) {
        // LOCK: Only one person enters here at a time
        pthread_mutex_lock(&myMutex);

        balance = balance + 1;

        // UNLOCK: Release for the next person
        pthread_mutex_unlock(&myMutex);
    }

    printf("%s finished depositing. Balance is now: %d\n", who, balance);
    return NULL;
}

int main() {
    pthread_t P1, P2;

    // Initialize the lock
    pthread_mutex_init(&myMutex, NULL);

    // Create two threads (Bintu and Bello)
    pthread_create(&P1, NULL, deposit, (void*)"bintu");
```

```
    pthread_create(&P2, NULL, deposit, (void*)"bello");

    // Wait for both to finish
    pthread_join(P1, NULL);
    pthread_join(P2, NULL);

    // Destroy the lock
    pthread_mutex_destroy(&myMutex);

    printf("Final Total Deposit: %d\n", balance);

    return 0;
}
```

Output



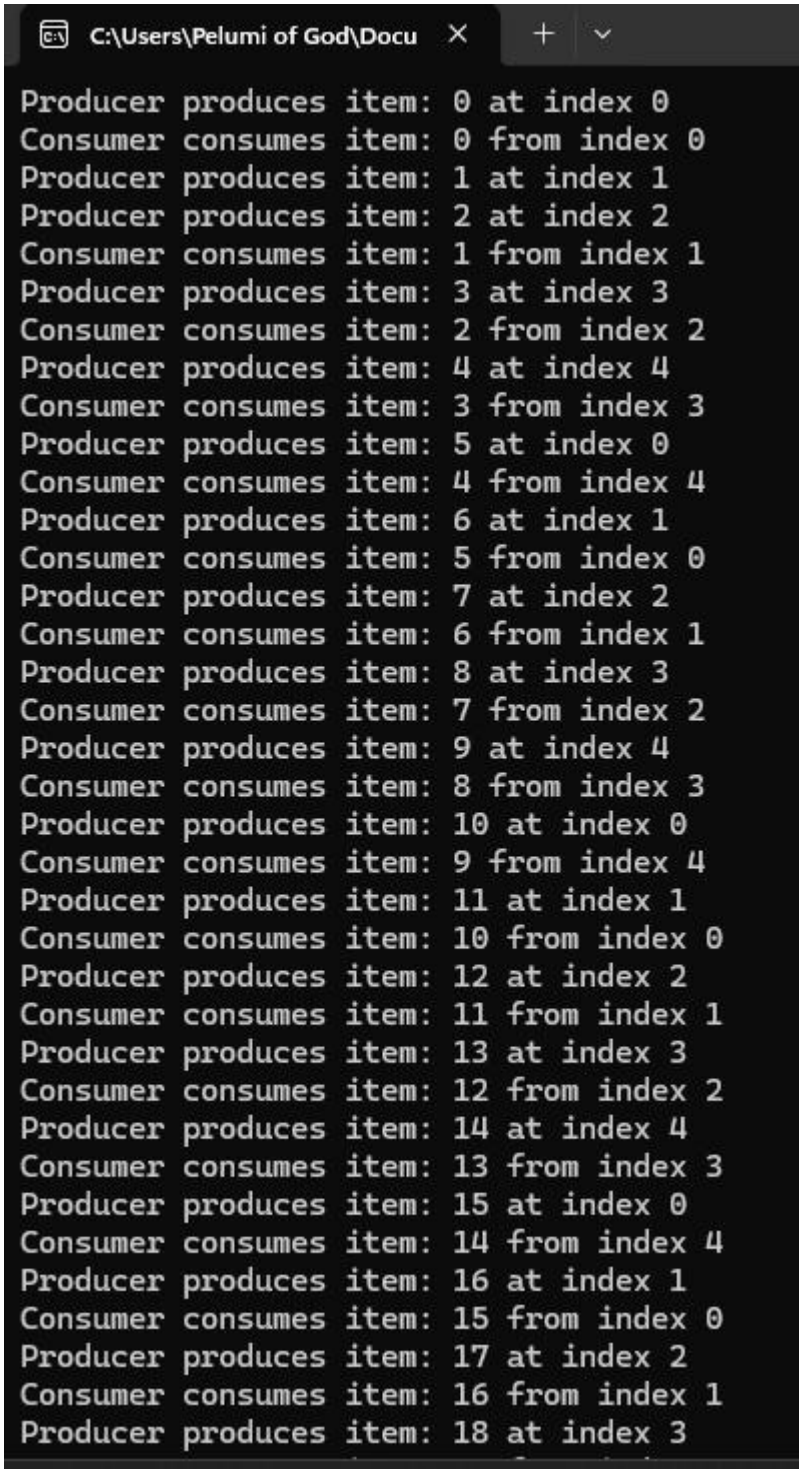
Question 3

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

```
sem_t mutex;
sem_t empty;
sem_t full;

void *producer(void *arg) {
    int item, i;
    for(i = 0; i < 20; i++) {
        item = i; // Produce an item (number 0 to 19)

        sem_wait(&empty); // Wait if buffer is full
        sem_wait(&mutex); // Lock critical section

        buffer[in] = item;
        printf("Producer produces item: %d at index %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;

        sem_post(&mutex); // Unlock
        sem_post(&full);  // Signal that there is new data
    }
    return NULL;
}

void *consumer(void *arg) {
    int item, i;
    for(i = 0; i < 20; i++) {

        sem_wait(&full);  // Wait if buffer is empty
        sem_wait(&mutex); // Lock critical section

        item = buffer[out];
        printf("Consumer consumes item: %d from index %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        sem_post(&mutex); // Unlock
        sem_post(&empty); // Signal that there is empty space
    }
```

```
    return NULL;
}

int main() {
    pthread_t prod, cons;

    // Initialize semaphores
    sem_init(&mutex, 0, 1);
    sem_init(&full, 0, 0);          // Initially 0 items are full
    sem_init(&empty, 0, BUFFER_SIZE); // Initially all slots are empty

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&mutex);
    sem_destroy(&full);
    sem_destroy(&empty);

    return 0;
}
```

Output


C:\Users\Pelumi of God\Docu
Producer produces item: 0 at index 0
Consumer consumes item: 0 from index 0
Producer produces item: 1 at index 1
Producer produces item: 2 at index 2
Consumer consumes item: 1 from index 1
Producer produces item: 3 at index 3
Consumer consumes item: 2 from index 2
Producer produces item: 4 at index 4
Consumer consumes item: 3 from index 3
Producer produces item: 5 at index 0
Consumer consumes item: 4 from index 4
Producer produces item: 6 at index 1
Consumer consumes item: 5 from index 0
Producer produces item: 7 at index 2
Consumer consumes item: 6 from index 1
Producer produces item: 8 at index 3
Consumer consumes item: 7 from index 2
Producer produces item: 9 at index 4
Consumer consumes item: 8 from index 3
Producer produces item: 10 at index 0
Consumer consumes item: 9 from index 4
Producer produces item: 11 at index 1
Consumer consumes item: 10 from index 0
Producer produces item: 12 at index 2
Consumer consumes item: 11 from index 1
Producer produces item: 13 at index 3
Consumer consumes item: 12 from index 2
Producer produces item: 14 at index 4
Consumer consumes item: 13 from index 3
Producer produces item: 15 at index 0
Consumer consumes item: 14 from index 4
Producer produces item: 16 at index 1
Consumer consumes item: 15 from index 0
Producer produces item: 17 at index 2
Consumer consumes item: 16 from index 1
Producer produces item: 18 at index 3